FAQ-бот Astana IT University

Как бот ищет ответ: пять методов для чайника

Rule-based, Bag of Words, pg_trgm, TF-IDF и Embeddings — от простого к сложному. Как устроен каждый метод в нашем проекте, на настоящих вопросах из базы и с настоящими цифрами.

Что делает бот

В базе 556 вопросов с ответами (FAQ о поступлении). Пользователь пишет свой вопрос, бот находит в FAQ самый похожий вопрос и отдаёт его ответ. Все пять методов решают одну задачу: **посчитать, насколько вопрос пользователя похож на каждый из 556 вопросов FAQ**. Отличаются они только тем, как считают похожесть.

У каждого метода свой **порог**: если лучший результат ниже порога, бот честно говорит «не нашёл».

Методы по сложности

#	Метод	Что сравнивает	Подготовка	Поиск	Память
1	Rule-based	общие слова + ручные правила синонимов	0.03 с	~0.5 мс	0.03 МБ
2	Bag of Words	сколько раз встретилось каждое слово	0.01 с	~1 мс	0.07 МБ
3	pg_trgm	кусочки по 3 буквы (считает PostgreSQL)	0 с	~12 мс	0.38 МБ
4	TF-IDF	слова с весом: редкое слово важнее частого	0.02 с	~1 мс	0.20 МБ
5	Embeddings	смысл фразы (нейросеть, 384 числа)	~9 с	~16 мс	450 МБ

Время и память измерены на нашей базе. «Подготовка» — один раз при запуске бота, «Поиск» — на каждый вопрос.

Перед тем как читать дальше

Как мы проверяем методы

Похожесть, которую выдаёт метод, ещё не значит, что ответ верный. Поэтому есть **тестовые вопросы**, для которых мы заранее знаем правильный ответ, и смотрим, на какое место метод его поставил.

Набор	Что внутри
Базовый (40)	по одной разговорной перефразе на 40 разных вопросов FAQ: «что за универ аиту», «как мне поступить в аиту»
Формулировки (64)	10 вопросов, у каждого 6–7 формулировок разного типа: сленг («военка»), синоним («цена обучения»), опечатка («стоимсть»), транслит («фаундейшн»), английский («tuition fee»), неполная фраза («контакты»), зеркальная («по АЕТ без ЕНТ» вместо «по ЕНТ без АЕТ»)

Метрика	Как считается	Пример
P@1	доля вопросов, где правильный ответ стоит первым	P@1 = 0.5 — половина ответов верны сразу
P@3	доля вопросов, где правильный ответ есть в первой тройке	можно показать пользователю 3 варианта
MRR	среднее от 1 / место правильного ответа: 1-е место = 1, 2-е = 0.5, 3-е = 0.33, нет в топ-3 = 0	главная метрика: учитывает, насколько близко к верху
Уверенная ошибка	неверный ответ, у которого похожесть выше порога метода	бот выдал бы неверный ответ как найденный

Общая предобработка текста

Rule-based, Bag of Words и TF-IDF сначала превращают текст в список «основ» слов, чтобы «обучение», «обучения» и «обучением» считались одним словом. pg_trgm только переводит текст в нижний регистр, Embeddings работают с исходным текстом.

```
Вопрос:            Сколько стоит обучение в АИТУ
1. нормализация    сколько стоит обучение в аиту          (нижний регистр, ё → е)
2. слова           [сколько] [стоит] [обучение] [в] [аиту]
3. стоп-слова      [сколько] [стоит] [обучение]     [аиту]   («в», «и», «ли», «есть» — убираются)
4а. стемминг       [скольк]  [сто]   [обучен]       [аит]    (отрезаем окончание по правилам)
4б. лемматизация   [сколько] [стоить] [обучение]    [аит]    (словарная форма — точнее, но медленнее)
```

Вопросительные слова («как», «сколько», «где») специально **не** считаются стоп-словами: в FAQ они несут смысл вопроса. Обратите внимание на ловушку: стеммер превратил «стоит» в **«сто»** (как число 100). Эта ошибка ещё встретится в примерах.

1 Rule-based

`methods/rule_based.py`

базовый метод	обработка: стемминг	порог 0.15	0.03 МБ · ~0.5 мс

> **Идея в одной фразе.** Сравниваем, сколько общих слов у вопроса пользователя и вопроса из FAQ, предварительно заменив сленг на нормальные слова по заранее написанным правилам.

Аналогия

Библиотекарь без интернета с 556 карточками. Вы спрашиваете «есть ли общага?». Он знает пару жаргонных слов и мысленно переводит «общага» → «общежитие», выбрасывает пустые слова «есть», «ли», перебирает карточки и для каждой прикидывает, насколько совпадает набор слов. Смысла не понимает — только сравнивает слова.

Правила (их всего 6)

Что пишет человек	Во что превращается
аиту, astana it university, астана ит университет	aitu
общага, общаги, общагу…	общежитие
универ, вуз	университет
стипуха	стипендия
цена, оплата, стоимость	стоимость
магистр…	магистратура

Как устроено у нас

Подготовка (один раз)	Поиск (на каждый вопрос)
1. К каждому вопросу FAQ применяются правила.	**1.** Те же правила и предобработка для запроса.
2. Обычная предобработка: слова, стоп-слова, стемминг.	**2.** С каждым из 556 вопросов считается коэффициент Жаккара.
3. Вопрос запоминается как **множество** слов — порядок и повторы не важны.	**3.** Берётся лучший; если меньше 0.15 — «не нашёл».

```
Жаккар = сколько слов общих / сколько разных слов всего в обоих вопросах      (от 0 до 1)
```

Пример, где работает

```
Запрос «есть ли общага»
  правило:  общага → общежитие          стоп-слова: «есть», «ли»
  слова:    {общежит}

  1.00  «Есть ли у вас общежитие?»              {общежит}                        1 общее из 1   ✓
  0.50  «Есть ли общежитие в AITU?»             {aitu, общежит}                  1 из 2
  0.25  «Сколько стоит проживание в общежитии?»  {общежит, проживан, скольк, сто}   1 из 4
```

Без правила слово «общаг» не совпало бы ни с одним вопросом — именно здесь Bag of Words и TF-IDF дают ноль.

Где не справляется

```
«есть военка?»  →  {военк}
  такого слова нет ни в одном вопросе FAQ, правила для «военки» нет  →  0.00 со всеми, «не нашёл»

«как получить место в общаге»  →  {как, мест, общежит, получ}
  0.33  «Какие категории студентов имеют приоритет при распределении мест в общежитии?»   ✗
  0.33  «Как студенты могут подать заявление на предоставление места в общежитии?»       ← правильный
```

Во втором примере сразу три слабости: **ничья** (победил тот, кто раньше в базе); **ошибка стемминга** («какие» → «как» совпало с «как»); **все слова весят одинаково** («как» значит столько же, сколько «общежитие»).

Плюсы и минусы

Сильные стороны	Слабые стороны
• самый быстрый и лёгкий, не нужно обучение	• знает только синонимы из правил — остальное дописывать руками
• всегда понятно, почему выбран ответ	• все слова одинаково важны
• правила ловят сленг, на котором ломаются BoW и TF-IDF	• не понимает перефразы: «стоимость» ≠ «сколько стоит»
	• ничьи решает порядок в базе

Цифры

Показатель	Значение	Что это значит
MRR, базовый набор	0.537 (лемм. 0.621)	**1-е место** с лемматизацией: тестовые вопросы разговорные, а сленг — то, что закрывают правила
MRR, формулировки	0.315 (лемм. 0.331)	с лемматизацией 2-е место после Embeddings; проваливается на синонимах и английском
Уверенные ошибки	62%	доля неверных ответов, выданных выше порога

Rule-based — **точка отсчёта** (baseline): если сложный метод его не обгоняет, сложность не окупается. Самый дешёвый способ его улучшить — дописать правила для частых ошибок: «военка», «пара», «доки», «приемка».

2 Bag of Words

`methods/bow.py`

«мешок слов»	обработка: стемминг	порог 0.3	0.07 МБ · ~1 мс

> **Идея в одной фразе.** Каждый вопрос превращаем в список чисел «сколько раз встретилось каждое слово» и ищем вопрос, у которого такой список больше всего похож на список запроса.

Аналогия

Высыпьте слова вопроса в мешок: порядок теряется, остаётся только состав. «Как поступить в АИТУ» и «в АИТУ поступить как» — один и тот же мешок. Мы ищем, в каком из 556 мешков FAQ лежат примерно те же слова в тех же пропорциях.

Чем отличается от Rule-based

	Rule-based	Bag of Words
Правила синонимов	есть («общага» → «общежитие»)	**нет**
Повтор слова	не важен (множество)	**считается** (2 раза = 2)
Сравнение	Жаккар: общих / всех	**косинус** между векторами

Как устроено у нас

Подготовка (один раз)	Поиск (на каждый вопрос)
1. Предобработка каждого вопроса FAQ.	**1.** Та же предобработка запроса.
2. **Словарь**: все разные основы слов из FAQ — у нас их **825**.	**2.** Вектор запроса длиной 825; слова, которых нет в словаре, **выбрасываются**.
3. **Матрица 556 × 825**: строка — вопрос, столбец — слово, в ячейке — сколько раз слово встретилось. В вопросе в среднем 7–8 слов, поэтому заполнено лишь **0.92%** ячеек.	**3.** Косинус с каждой из 556 строк; лучший — ответ, если не меньше 0.3.

```
cos = (сумма произведений совпавших чисел) / (длина вектора запроса × длина вектора вопроса)
длина вектора = √(сумма квадратов чисел)                                  (от 0 до 1)
```

Пример, где работает

```
«как поступить в аиту»  →  [как:1, поступ:1, аит:1]
FAQ «Как поступить в Astana IT University (AITU / аиту)?»
    →  [как, поступ, astana, it, university, aitu, аит] — 7 слов по 1

общих: как, поступ, аит   →  1+1+1 = 3
длина запроса √3 ≈ 1.73,  длина вопроса √7 ≈ 2.65
cos = 3 / (1.73 × 2.65) = 0.655   ✓
```

С тем же 0.655 идёт «Как поступить в колледж АИТУ?» — победил порядок в базе, а не смысл.

Где не справляется

```
«сколько стоит обучение в аиту»  →  [скольк, сто, обучен, аит]
  0.50  «Сколько стоит проживание в общежитии?»   общих: скольк, сто          2 / (2 × 2) = 0.50  ✗
  0.50  «Сколько курсы по английскому стоят?»     общих: скольк, сто          2 / (2 × 2) = 0.50  ✗
  0.50  «Сколько лет длится обучение в AITU?»     общих: скольк, обучен, аит  3 / (2 × 3) = 0.50  ✗

«есть ли общага»  →  [общаг]  — такого слова нет в словаре  →  0.00 со всеми
```

Три разных вопроса с одинаковым счётом, и ни один не правильный: «стоимость» и «стоит» для метода — разные слова. Ещё нюанс: без правил «AITU» и «аиту» тоже разные слова (`aitu` и `аит`).

Плюсы и минусы

Сильные стороны	Слабые стороны
• стандартный и понятный, считается руками	• **все слова равноценны**: «сколько» весит как «обучение»
• быстрый и лёгкий	• незнакомое слово (сленг, опечатка, транслит) просто пропадает
• учитывает повторы слов	• не знает синонимов
	• частые ничьи

Цифры

Показатель	Значение	Что это значит
MRR, базовый набор	0.538 (лемм. 0.592)	почти как Rule-based: правил нет, но есть косинус
MRR, формулировки	0.263	**последнее место**: ни правил для сленга, ни понимания смысла
Уверенные ошибки	72%	худший показатель — чаще всех ошибается уверенно

Главную слабость — равенство всех слов — исправляет TF-IDF: редкие слова получают больший вес.

3 pg_trgm

methods/pg_trgm.py · считает PostgreSQL

кусочки по 3 буквы	обработка: нижний регистр	порог 0.3	0.38 МБ · ~12 мс

> **Идея в одной фразе.** Сравниваем не слова, а кусочки по три буквы (триграммы): чем больше кусочков запроса нашлось в вопросе, тем он похожее. Весь расчёт делает база данных.

Аналогия

Узнавать слово по обрывкам. В опечатке «стоимсть» большинство обрывков такие же, как в «стоимость», поэтому слово всё равно узнаётся. А «общага» и «общежитие» начинаются одинаково («общ…»), и это уже подсказка — хотя целиком слова разные.

Чем отличается от предыдущих

Единица сравнения — не слово, а 3 символа. Нет стоп-слов и стемминга: текст только переводится в нижний регистр. Индекс в Python не строится — вопросы уже лежат в таблице `faq`, а функцию сходства даёт расширение PostgreSQL `pg_trgm`.

Как устроено у нас

Подготовка (один раз)	Поиск — один SQL-запрос на каждый вопрос
1. Ничего не строится: запоминаем только, в каком порядке идут вопросы, чтобы разложить оценки.	**1.** Запрос и вопрос → нижний регистр. **2.** Каждое слово дополняется пробелами (2 спереди, 1 сзади) и режется на тройки букв. **3.** `word_similarity`: доля триграмм запроса, найденных в **самом похожем фрагменте** вопроса. **4.** Лучший — ответ, если не меньше 0.3.

```
«общага»     →  "  о"  " об"  "общ"  "бща"  "щаг"  "ага"  "га "                    7 триграмм
«общежитие»  →  "  о"  " об"  "общ"  "бще"  "щеж"  "ежи"  "жит"  "ити"  "тие"  "ие "
общих: "  о", " об", "общ"   →   word_similarity = 3 / 7 = 0.43

SELECT faq_id, word_similarity(lower(:запрос), lower(question)) FROM faq
```

Длинный вопрос не штрафуется за лишние слова: ищется самый похожий кусок, а не сравнивается всё целиком.

Пример, где работает

```
«стоимсть обучения» (опечатка)   →  0.76  «Какова стоимость обучения в университете?»   ✓
                                        (без опечатки было бы 1.00 — потеряли только пару триграмм)
«есть ли общага»                 →  0.73  «Есть ли общежитие в AITU?»                    ✓
```

Где не справляется

```
«сколько стоит обучение в аиту»  →  0.50  «Сколько стоит проживание в общежитии?»   ✗
                                          совпали буквы фразы «сколько стоит», смысл другой
«что такое фаундейшн»            →  0.50  «Что такое AITU?» / «Что такое IoT?» / «Что такое
                                          машинное обучение?» — ничья, совпало только «что такое»
«AITU» и «аиту»                  →  0.00  разные алфавиты — ни одной общей триграммы
```

Плюсы и минусы

Сильные стороны	Слабые стороны
• **лучший на опечатках и обрывках** («поступить» / «поступление» делят много триграмм)	• не понимает смысл и синонимы
• не нужен индекс в Python — всё делает база	• латиница и кириллица — разные строки
• не нужны правила и стемминг	• на каждый вопрос обращается к базе и перебирает все строки: ~12 мс, в 10–20 раз медленнее словесных методов

Цифры

Показатель	Значение	Что это значит
MRR, базовый набор	0.429	последнее место: разговорные перефразы меняют слова целиком
MRR, формулировки	0.307	4-е место
P@1 на опечатках	0.50	**лучший** среди всех методов (у остальных 0.17)
P@1 на неполных фразах	0.57	**лучший** («сроки приема», «контакты»)
Уверенные ошибки	61%	

4 TF-IDF

`methods/tfidf.py`

веса слов	обработка: стемминг	порог 0.3	0.20 МБ · ~1 мс

> **Идея в одной фразе.** Как Bag of Words, но у каждого слова свой вес: слово, которое встречается почти везде, почти ничего не значит, а редкое слово — сильная подсказка.

Аналогия

Если слово «как» написано на каждой второй карточке, по нему ничего не найти. Если слово «общежитие» есть только на десяти карточках из 556 — оно сразу сужает поиск. TF-IDF присваивает словам вес по этому принципу.

Чем отличается от Bag of Words

	Bag of Words	TF-IDF
Вес слова	все слова равны	**редкое слово весит больше** (IDF)
Повтор слова	2 раза = 2	1 + ln(повторов): повтор почти не раздувает вес
Что в словаре	слова (825)	слова **и пары соседних слов** — биграммы (2914 терминов)

Как устроено у нас

Подготовка (один раз)	Поиск (на каждый вопрос)
1. Предобработка каждого вопроса FAQ.	**1.** Та же предобработка запроса.
2. Словарь из слов и биграмм: «скольк», «сто», «скольк сто».	**2.** Вектор запроса с теми же IDF.
3. **df** — в скольких вопросах из 556 есть термин.	**3.** Косинус с каждым вопросом; лучший — ответ, если не меньше 0.3.
4. **IDF** = ln((1 + 556) / (1 + df)) + 1 — чем реже, тем больше.	
5. Вес = (1 + ln повторов) × IDF; вектор нормируется до длины 1.	

Веса из нашей базы

Термин	В скольких вопросах (df)	IDF	Вывод
как	325 из 556	1.54	почти везде — почти ничего не значит
обучен	59	3.23	частое
аит	47	3.45	частое
скольк	15	4.55	редкое
общежит	10	4.92	редкое — сильная подсказка
сто	3	5.94	очень редкое (и это ошибка стемминга «стоит» → «сто»)
скольк сто	1	6.63	биграмма — есть ровно в одном вопросе

Пример, где работает

«как поступить в аиту» → 0.49 «Как поступить в AITU?» ✓. Слово «как» почти ничего не весит, решают «поступ» и «аит». В отличие от BoW, у второго места («Что нужно сдать чтобы поступить в АИТУ?», 0.47) уже не ничья.

Где не справляется

```
«сколько стоит обучение в аиту»  →  [скольк, сто, обучен, аит] + биграммы
  самый тяжёлый термин запроса — «скольк сто» (IDF 6.63), и он есть только в одном вопросе:
  0.586  «Сколько стоит проживание в общежитии?»   ✗     (у BoW тут была ничья 0.50 / 0.50 / 0.50)
  0.312  «Сколько курсы по английскому стоят?»
  0.218  «Сколько лет длится обучение в AITU?»
```

Метод правильно нашёл самое редкое совпадение — но не понял, что спрашивали про обучение. А «обучен» встречается в 59 вопросах и весит мало. Ошибка стемминга («сто») прямо усиливает неверный ответ.

Плюсы и минусы

Сильные стороны

- важные слова решают, частые почти не мешают
- биграммы ловят устойчивые фразы («проходной балл»)
- меньше ничьих, чем у BoW
- те же веса можно посчитать SQL-запросом (tfidf_keywords)

Слабые стороны

- не понимает смысл: «стоимость обучения» и «сколько стоит учиться» — разные слова
- редкое, но случайное совпадение перевешивает
- ошибки стемминга влияют на веса

Цифры

Показатель	Значение	Что это значит
MRR, базовый набор	0.525 (лемм. 0.596)	с лемматизацией 2-е место
MRR, формулировки	0.266	6-е место
P@1 на перестановке слов	0.67	**лучший** — биграммы и веса помогают на «можно только по ЕНТ без АЕТ»
Уверенные ошибки	53%	реже, чем у BoW и Rule-based

5 Embeddings

methods/embeddings.py

нейросеть	обработка: нет, исходный текст	порог 0.5	450 МБ · ~16 мс

> **Идея в одной фразе.** Готовая нейросеть переводит смысл фразы в 384 числа. Близкие по смыслу фразы получают близкие числа, даже если у них нет ни одного общего слова.

Аналогия

Представьте карту смыслов, где каждая фраза — точка. «Сколько стоит обучение», «какова стоимость обучения» и «tuition fee» лежат рядом, а «сколько стоит проживание» — в другом районе, хотя слов общих больше. Метод ищет вопрос FAQ, чья точка ближе всего к точке запроса.

Чем отличается от предыдущих

Слова больше не считаются. Модель `paraphrase-multilingual-MiniLM-L12-v2` (около 118 млн параметров) **уже обучена** на миллионах пар перефраз на 50+ языках. Мы её не обучаем, только применяем. Стоп-слова и стемминг не нужны — модель сама разбирает текст.

Как устроено у нас

Подготовка (один раз, ~9 с)	Поиск (на каждый вопрос, ~16 мс)
1. Загрузка модели (450 МБ памяти — почти всё это веса нейросети).	**1.** Запрос → вектор той же моделью.
2. Все 556 вопросов → матрица **556 × 384**; каждый вектор нормируется до длины 1.	**2.** Сходство = скалярное произведение (для векторов длины 1 это и есть косинус).
	3. Лучший — ответ, если не меньше 0.5 (у нейросети оценки в целом выше, поэтому порог выше).

```
«сколько стоит обучение»  →  [0.082, −0.016, −0.008, 0.018, −0.007, 0.008, … ещё 378 чисел]
По отдельности числа ничего не значат — важно только, насколько два вектора смотрят в одну сторону.
```

Пример, где работает

```
сходство «сколько стоит обучение» и «Какова стоимость обучения в университете?»  = 0.76   ✓
сходство «сколько стоит обучение» и «Сколько стоит проживание в общежитии?»      = 0.42
сходство «tuition fee»            и «Какова стоимость обучения в университете?»  = 0.77   (английский!)

«сколько стоит обучение в аиту»  →  0.74  «Какова стоимость обучения в университете?»  ✓
                                     единственный из пяти методов, нашедший правильный ответ
«tuition fee aitu»               →  0.72  тот же правильный ответ
```

Где не справляется

```
«есть ли общага»   →  0.52  «Есть пробники по АЕТ?»          ✗  (выше порога 0.5 — уверенная ошибка)
                      0.39  «Есть ли у вас общежитие?»       ← правильный, только 3-й
                      короткий сленговый запрос: модель «общагу» почти не знает

«можно поступить только по ент без ает»  →  0.58  «Можно ли без ЕНТ поступить, но по АЕТ?»  ✗
«можно поступить только по ает без ент»  →  0.62  «Можно ли без ЕНТ поступить, но по АЕТ?»
                      оба направления ведут в один вопрос: смысл «без чего» модель не различает
```

Плюсы и минусы

Сильные стороны	Слабые стороны
• **понимает синонимы и перефразы**	• тяжёлый: 450 МБ и ~9 с на загрузку
• работает на разных языках, включая английский	• в 10–30 раз медленнее словесных методов
• не нужны правила и предобработка	• не видно, **почему** выбран ответ
	• плохо понимает сленг и очень короткие запросы

Цифры

Показатель	Значение	Что это значит
MRR, базовый набор	0.463	7-е место из 8: перефразы там разговорные, их лучше ловят правила
MRR, формулировки	0.419	**1-е место**: только он справляется с синонимами и английским
P@1 на английском	0.50	у остальных 0–0.20
P@1 на разговорных	0.29	единственный ненулевой результат
Уверенные ошибки	52%	лучший показатель, но всё равно каждая вторая ошибка — уверенная

Итоги

Какой метод на каком типе формулировок лучше

P@1 на наборе из 64 формулировок (варианты со стеммингом). Жирным — лучший в строке.

Тип	Rule-based	BoW	pg_trgm	TF-IDF	Embeddings
сленг («военка», «общага»)	0.33	0.33	0.33	0.33	**0.67**
синоним («цена обучения»)	**0.26**	0.16	0.16	0.10	**0.26**
опечатка («стоимсть»)	0.17	0.17	**0.50**	0.17	0.17
неполная («контакты»)	0.29	0.29	**0.57**	0.29	0.29
порядок слов	0.33	0.33	0.33	**0.67**	0.33
английский («tuition fee»)	0.20	0.00	0.20	0.20	**0.50**
разговорная («сколько длится пара»)	0.00	0.00	0.00	0.00	**0.29**
транслит («фаундейшн»)	0.00	0.00	0.00	0.00	0.00

Зеркальные запросы («по АЕТ без ЕНТ») в таблицу не включены: почти все методы отвечают на оба направления одним и тем же вопросом, поэтому их «высокий» результат — совпадение, а не понимание.

Главные выводы

• **Сложнее не значит лучше.** На разговорных перефразах первое место у самого простого Rule-based с лемматизацией (MRR 0.621), а нейросеть — седьмая (0.463).

• **У каждого метода своя ниша.** Rule-based — сленг из правил, pg_trgm — опечатки и обрывки, TF-IDF — порядок слов и устойчивые фразы, Embeddings — синонимы и английский.

• **Методы часто ошибаются уверенно.** От 52% до 72% неверных ответов выдаются выше порога — пороги стоит подбирать заново.

• **Лемматизация лучше стемминга** для всех словесных методов (+0.05…0.08 MRR): стеммер портит слова вроде «стоит» → «сто».

• **Транслит не решает никто.** «фаундейшн», «айлтс» — ноль у всех методов; нужны правила или словарь транслитерации.

• Отсюда идея для бота: **комбинировать методы** — например, брать ответ, за который голосует больше методов, или сначала проверять опечатки триграммами, а смысл — нейросетью.

Все примеры и цифры получены на базе проекта: 556 вопросов FAQ, 40 + 64 тестовых вопроса, прогон 5 ноября 2026. Время поиска зависит от компьютера.